Reporte

EQUIPO 4	
García Cerda Randal Guillermo (2112493 ITS) Sarmiento Hernández Cesar Martín (2119427 ITS) Torres Pasillas Angela Susana (2101844 ITS)	
Nombre del curso: Componentes de Lógica de Negocios	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 4	**Actividad**: Unidad 4.
Fecha: 27/10/2026	
Bibliografia (Formato APA): Notas de clase	

Objetivo:

El estudiante implementará los componentes de lógica de negocio diseñados en U3: entidades DDD, reglas con Specification Pattern, máquinas de estado, Domain Events y APIs REST.

Procedimiento y desarrollo

El desarrollo siguió el orden de la práctica guiada: primero el dominio, después las Specifications, la máquina de estados con Domain Events y, al final, la API REST. Las reglas de negocio viven en app/domain.py, app/specifications.py y app/pricing.py; app/api.py solo las expone por HTTP. Los archivos de datos y cargar_csv.py son el material original del profesor y no se modificaron (una prueba compara la huella SHA-256 de cada CSV).

1.1 Entidades y Value Objects

Clase	Tipo	Responsabilidad e invariantes
Money	Value Object inmutable	amount (Decimal) y currency (MXN por defecto). Rechaza float, negativos, NaN e infinito. add() exige la misma moneda y devuelve un objeto nuevo.
Customer	Datos inmutables	id, is_vip y active. El identificador no puede estar vacío.
OrderItem	Value Object inmutable	sku, quantity y unit_price. SKU no vacío y cantidad entera > 0. El subtotal se calcula, no se captura.
OrderStatus	Enumeración	DRAFT, CONFIRMED, SHIPPED, DELIVERED, CANCELLED, RETURNED.
Order	Entidad / Aggregate Root	Inicia en DRAFT. units suma cantidades y total suma subtotales. add_item() solo en DRAFT, solo MXN y máximo 20 unidades acumuladas; valida antes de modificar.

Excepción	Significado	Ejemplo real	HTTP
DomainException	Base de las violaciones de reglas	—	—
InvalidOrderData	Datos o precondiciones inválidos	Confirmar O007 (vacío) u O003 (cliente inactivo); enviar sin guía	422
InvalidOrderTransition	Operación incompatible con el estado	Confirmar O013 dos veces; cancelar O014 ya enviado	409
OrderItemLimitExceeded	Se superarían 20 unidades	Agregar 2 unidades a O009 (19 + 2 = 21)	422

1.2 Specification Pattern y descuentos

Una Specification convierte una condición en un objeto con is_satisfied_by(order) que devuelve True o False sin modificar el pedido. Se combinan con & (AND), | (OR) y ~ (NOT).

Regla	Condiciones	Specifications
Descuento 10 %	Cliente VIP, total > 500 MXN, al menos 4 unidades y sin promoción	MinimumAmountSpec(500) & VIPCustomerSpec() & MinimumUnitsSpec(4) & ~ActivePromotionSpec()
Descuento 5 %	Total > 5000 MXN y sin promoción	MinimumAmountSpec(5000) & ~ActivePromotionSpec()
Combinación	Si se cumplen ambas, se aplica primero el 10 %	TEN_PERCENT_RULE | FIVE_PERCENT_RULE; el monto lo calcula app/pricing.py

1.3 Máquina de estados

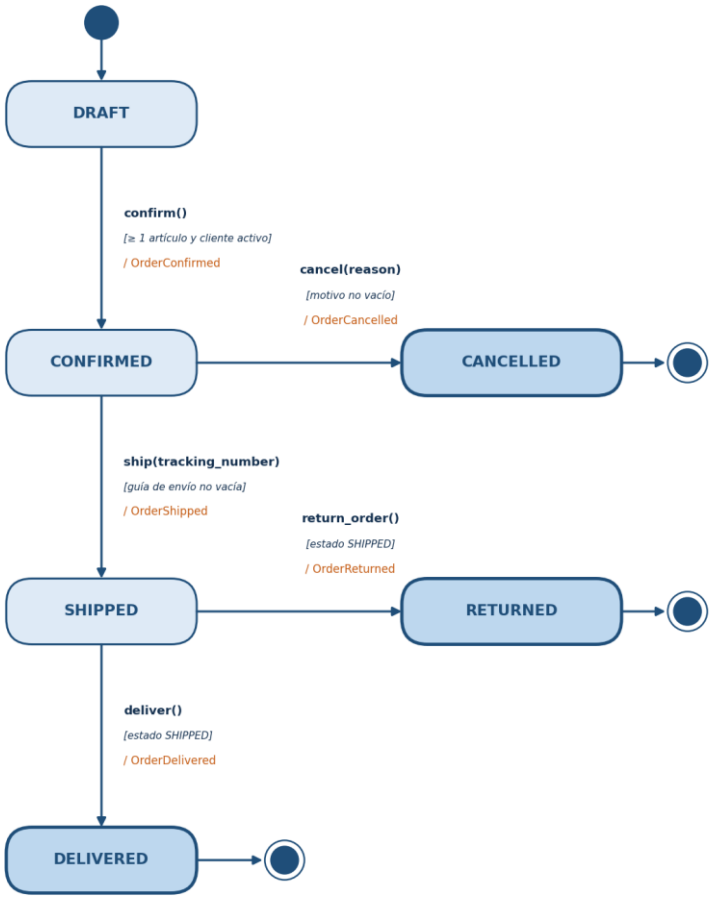
Transición	Método	Validaciones previas
DRAFT → CONFIRMED	confirm()	Estado DRAFT, pedido con artículos y cliente activo
CONFIRMED → SHIPPED	ship(tracking_number)	Estado CONFIRMED y guía no vacía
CONFIRMED → CANCELLED	cancel(reason)	Estado CONFIRMED y motivo no vacío
SHIPPED → DELIVERED	deliver()	Estado SHIPPED
SHIPPED → RETURNED	return_order()	Estado SHIPPED

Cualquier otra combinación lanza InvalidOrderTransition (25 combinaciones inválidas de 30, todas cubiertas por pruebas). Los métodos siguen siempre el orden: validar, cambiar el estado y registrar el evento. El diagrama de la Figura 1 representa exactamente estas cinco transiciones.

Máquina de estados del pedido TechStore

clase Order · app/domain.py · TechStore U4 (Componentes de lógica de negocios)

notación: operación() [condición] / evento registrado



DRAFT (estado inicial)

- Único estado que admite add_item(): máximo 20 unidades acumuladas (suma de cantidades, no de renglones), en MXN.
- confirm() sin artículos o con cliente inactivo lanza InvalidOrderData (HTTP 422); el pedido sigue en DRAFT.

CONFIRMED

- ship('') lanza InvalidOrderData (422): sigue CONFIRMED y no se registra OrderShipped.
- cancel('') lanza InvalidOrderData (422): sin evento.

SHIPPED

- Solo admite deliver() o return_order().
- cancel() lanza InvalidOrderTransition (409): sigue SHIPPED y no se registra OrderCancelled.

Reglas comunes

1) Transición válida: valida → cambia estado → registra evento.

2) Operación no permitida: InvalidOrderTransition (409), sin cambiar el estado ni registrar eventos (p. ej. confirmar O013 dos veces; cancelar O014 ya enviado).

3) DELIVERED, RETURNED y CANCELLED son estados finales.

1.4 Domain Events

OrderConfirmed, OrderShipped, OrderCancelled, OrderDelivered y OrderReturned heredan de DomainEvent (name, order_id y occurred_at en UTC). OrderShipped guarda además tracking_number y OrderCancelled guarda reason. Los eventos solo se registran después de una transición exitosa; una operación rechazada no deja cambios ni eventos, como se comprueba con la segunda confirmación de O013 (E12).

Evento	Lo emite	Datos que lleva
OrderConfirmed	confirm()	order_id, occurred_at
OrderShipped	ship(tracking_number)	order_id, occurred_at, tracking_number
OrderCancelled	cancel(reason)	order_id, occurred_at, reason
OrderDelivered	deliver()	order_id, occurred_at
OrderReturned	return_order()	order_id, occurred_at

1.5 API REST con FastAPI

Método	Ruta	Respuestas
GET	/orders/{id}	200; 404
POST	/orders/{id}/confirm	200; 404; 409; 422
POST	/orders/{id}/ship (cuerpo: tracking_number)	200; 404; 409; 422
POST	/orders/{id}/cancel (cuerpo: reason)	200; 404; 409; 422
POST	/orders/{id}/deliver · /orders/{id}/return (extra)	200; 404; 409
GET	/orders · /orders/{id}/discount (extra)	200; 404

Los pedidos se mantienen en memoria, cargados con load_fixture(); no hay base de datos. Los controladores solo buscan el pedido y llaman a su método; los exception handlers traducen InvalidOrderTransition a 409, el resto de DomainException a 422 y un pedido inexistente a 404. Los errores inesperados devuelven 500 sin detalles internos. Swagger queda en /docs y el contrato en /openapi.json.

2. Casos de estudio

2.1 Caso O001: carga, total y descuento del 10 %

O001 pertenece a C001 (VIP activo) y contiene SKU001 × 2 (600.00) y SKU002 × 2 (450.00). Unidades: 4. Subtotales: 1200.00 + 900.00 = 2100.00 MXN. Cumple VIP, total > 500, 4 unidades (≥ 4) y sin promoción, por lo que recibe 10 %: descuento 210.00 MXN y total final 1890.00 MXN (escenario E01).

2.2 Caso O014: por qué no se cancela después del envío

O014 se confirma y se envía (SHIPPED). La regla exige que cancel() solo opere desde CONFIRMED, así que el intento lanza InvalidOrderTransition (409 en la API). El pedido queda en SHIPPED con dos eventos (OrderConfirmed y OrderShipped) y no se añade OrderCancelled (escenario E13).

2.3 Caso O013: la confirmación doble

La primera confirmación pasa DRAFT → CONFIRMED y registra un OrderConfirmed. La segunda se rechaza porque el pedido ya no está en DRAFT: lanza InvalidOrderTransition, el estado sigue en CONFIRMED y la lista de eventos conserva un solo evento (E12).

2.4 Casos frontera: O010 y O009

O010 vale exactamente 500.00 MXN: no supera el umbral estricto, por eso no recibe descuento (E09). O009 tiene 19 unidades en un solo renglón; agregar 2 llegaría a 21 y se rechaza sin modificar el pedido (E08): el límite se mide en unidades, no en renglones.

3. Pruebas y validaciones

3.1 Resultados de la ejecución

Archivo	Casos	Qué cubre
tests/test_domain.py	81	Money, OrderItem, Order, límite de 20 unidades, transiciones válidas e inválidas, eventos, descuentos
tests/test_api.py	22	GET, confirm, ship, cancel, deliver/return, 404, 409, 422, /docs y /openapi.json
tests/test_specifications.py	39	Operadores &, | y ~ (tablas de verdad), cada Specification, fronteras, prioridad del 10 %, redondeo y ausencia de efectos secundarios
tests/test_datos.py	22	Huellas SHA-256 de los CSV originales, conteo de registros, 16 pedidos en DRAFT, O001 = 4 unidades y 2100.00 MXN
tests/test_escenarios.py	17	Los 15 escenarios del CSV con estado final y eventos, más aislamiento entre casos

Resultado de la verificación. Con pytest real, python -m pytest -q ejecutó los cinco archivos de pruebas y terminó con 181 passed y 1 advertencia de deprecación de httpx (no es una falla), como se muestra en la Figura 4.

3.2 Los 15 escenarios

python ejecutar_escenarios.py ejecuta E01–E15 sobre objetos cargados con load_fixture() (una carga nueva por escenario) y termina con 15/15 escenarios que cumplen su objetivo. No basta con que el programa no falle: cada caso se contrasta con la columna objetivo del CSV, con la operación donde debía ocurrir el rechazo, con el estado final y con los eventos.

4. Bitácora completa de los escenarios E01–E15

Caso	Estado inicial	Operación	Resultado real	Explicación
E01	DRAFT	O001: evaluar_descuento	Descuento 10% (subtotal 2100.00, descuento 210.00, total con descuento 1890.00); estado final DRAFT; unidades 4; total 2100.00 MXN; eventos ninguno	PASA. O001: cliente C001 VIP=True, promoción=False, 4 unidades, total 2100.00 MXN. Esperado 10%, obtenido 10%. Evaluar no modifica el pedido (sigue en DRAFT, sin eventos).
E02	DRAFT	O002: evaluar_descuento	Descuento 0% (subtotal 1050.00, descuento 0.00, total con descuento 1050.00); estado final DRAFT; unidades 2; total 1050.00 MXN; eventos ninguno	PASA. O002: cliente C002 VIP=False, promoción=False, 2 unidades, total 1050.00 MXN. Esperado 0%, obtenido 0%. Evaluar no modifica el pedido (sigue en DRAFT, sin eventos).
E03	DRAFT	O003: confirmar	confirm rechazado (InvalidOrderData: No se puede confirmar: cliente inactivo); estado final DRAFT; unidades 4; total	PASA. La operación inválida se rechazó con InvalidOrderData y no dejó cambios parciales ni eventos extra (True). Pista del CSV: Cliente inactivo.

Caso	Estado inicial	Operación	Resultado real	Explicación
			2100.00 MXN; eventos ninguno	
E04	DRAFT	O005: evaluar_descuento	Descuento 0% (subtotal 2100.00, descuento 0.00, total con descuento 2100.00); estado final DRAFT; unidades 4; total 2100.00 MXN; eventos ninguno	PASA. O005: cliente C005 VIP=True, promoción=True, 4 unidades, total 2100.00 MXN. Esperado 0%, obtenido 0%. Evaluar no modifica el pedido (sigue en DRAFT, sin eventos).
E05	DRAFT	O006: evaluar_descuento	Descuento 5% (subtotal 18000.00, descuento 900.00, total con descuento 17100.00); estado final DRAFT; unidades 1; total 18000.00 MXN; eventos ninguno	PASA. O006: cliente C006 VIP=False, promoción=False, 1 unidades, total 18000.00 MXN. Esperado 5%, obtenido 5%. Evaluar no modifica el pedido (sigue en DRAFT, sin eventos).
E06	DRAFT	O007: confirmar	confirm rechazado (InvalidOrderData: No se puede confirmar un pedido vacío); estado final DRAFT; unidades 0; total 0.00 MXN; eventos ninguno	PASA. La operación inválida se rechazó con InvalidOrderData y no dejó cambios parciales ni eventos extra (True). Pista del CSV: Pedido sin articulos.

Caso	Estado inicial	Operación	Resultado real	Explicación
E07	DRAFT	O008: confirmar	confirm OK; estado final CONFIRMED; unidades 20; total 2400.00 MXN; eventos ['OrderConfirmed']	PASA. Todas las operaciones fueron válidas; estado final CONFIRMED. Pista del CSV: 20 unidades permitidas.
E08	DRAFT	O009: agregar SKU003 x2	add_item rechazado (OrderItemLimitExceeded: Máximo 20 unidades); estado final DRAFT; unidades 19; total 2280.00 MXN; eventos ninguno	PASA. Había 19 unidades; +2 = 21 > 20. El límite se mide en unidades acumuladas. Sin cambios parciales: True.
E09	DRAFT	O010: evaluar_descuento	Descuento 0% (subtotal 500.00, descuento 0.00, total con descuento 500.00); estado final DRAFT; unidades 2; total 500.00 MXN; eventos ninguno	PASA. O010: cliente C010 VIP=False, promoción=False, 2 unidades, total 500.00 MXN. Esperado 0%, obtenido 0%. Evaluar no modifica el pedido (sigue en DRAFT, sin eventos).
E10	DRAFT	O011: evaluar_descuento	Descuento 0% (subtotal 1500.00, descuento 0.00, total con descuento 1500.00); estado final DRAFT; unidades 3; total	PASA. O011: cliente C011 VIP=True, promoción=False, 3 unidades, total 1500.00 MXN. Esperado 0%, obtenido 0%. Evaluar no

Caso	Estado inicial	Operación	Resultado real	Explicación
			1500.00 MXN; eventos ninguno	modifica el pedido (sigue en DRAFT, sin eventos).
E11	DRAFT	O012: confirmar y enviar sin guia	confirm OK; ship rechazado (InvalidOrderData: La guía de envío no puede estar vacía); estado final CONFIRMED; unidades 1; total 850.00 MXN; eventos ['OrderConfirmed']	PASA. La operación inválida se rechazó con InvalidOrderData y no dejó cambios parciales ni eventos extra (True). Pista del CSV: La confirmacion es valida pero falta guia.
E12	DRAFT	O013: confirmar dos veces	confirm OK; confirm rechazado (InvalidOrderTransition: Solo se confirma un pedido en DRAFT); estado final CONFIRMED; unidades 1; total 600.00 MXN; eventos ['OrderConfirmed']	PASA. La operación inválida se rechazó con InvalidOrderTransition y no dejó cambios parciales ni eventos extra (True). Pista del CSV: No permite confirmar desde CONFIRMED.
E13	DRAFT	O014: confirmar, enviar y cancelar	confirm OK; ship OK; cancel rechazado (InvalidOrderTransition: Solo se cancela	PASA. La operación inválida se rechazó con InvalidOrderTransition y no dejó cambios parciales ni eventos extra (True). Pista

Caso	Estado inicial	Operación	Resultado real	Explicación
			un pedido CONFIRMED); estado final SHIPPED; unidades 1; total 600.00 MXN; eventos ['OrderConfirmed', 'OrderShipped']	del CSV: No permite cancelar desde SHIPPED.
E14	DRAFT	O015: confirmar, enviar y devolver	confirm OK; ship OK; return_order OK; estado final RETURNED; unidades 1; total 450.00 MXN; eventos ['OrderConfirmed', 'OrderShipped', 'OrderReturned']	PASA. Todas las operaciones fueron válidas; estado final RETURNED. Pista del CSV: Termina en RETURNED.
E15	DRAFT	O016: confirmar, enviar y entregar	confirm OK; ship OK; deliver OK; estado final DELIVERED; unidades 1; total 2100.00 MXN; eventos ['OrderConfirmed', 'OrderShipped', 'OrderDelivered']	PASA. Todas las operaciones fueron válidas; estado final DELIVERED. Pista del CSV: Termina en DELIVERED.

5. Evidencias

Evidencia 1: Carga de los 16 pedidos originales

Salida de python cargar_csv.py: los 16 pedidos cargan en DRAFT; O001 muestra 4 unidades y 2100.00 MXN.

```
(.venv) PS C:\Users\anngi\Downloads\techstore-u4\techstore-u4> python cargar_csv.py
O001 DRAFT 4 2100.00
O002 DRAFT 2 1050.00
O003 DRAFT 4 2100.00
O004 DRAFT 1 850.00
O005 DRAFT 4 2100.00
O006 DRAFT 1 18000.00
O007 DRAFT 0 0.00
O008 DRAFT 20 2400.00
O009 DRAFT 19 2280.00
O010 DRAFT 2 500.00
O011 DRAFT 3 1500.00
O012 DRAFT 1 850.00
O013 DRAFT 1 600.00
O014 DRAFT 1 600.00
O015 DRAFT 1 450.00
O016 DRAFT 1 2100.00
```

Figura 2. Carga de los 16 pedidos con cargar_csv.py.

Evidencia 2: Escenarios E01–E15

```
PS C:\Users\anngi\Downloads\EQ_4_UNIDAD4_CLNEG\EQ#4_UNIDAD4_CLNEG> python ejecutar_escenarios.py
E01 PASA - evaluar_descuento -> 10%
E02 PASA - evaluar_descuento -> 0%
E03 PASA - confirmar -> rechazo
E04 PASA - evaluar_descuento -> 0%
E05 PASA - evaluar_descuento -> 5%
E06 PASA - confirmar -> rechazo
E07 PASA - confirmar -> exito
E08 PASA - agregar SKU003 x2 -> rechazo
E09 PASA - evaluar_descuento -> 0%
E10 PASA - evaluar_descuento -> 0%
E11 PASA - confirmar y enviar sin guia -> rechazo envio
E12 PASA - confirmar dos veces -> rechazo segunda
E13 PASA - confirmar, enviar y cancelar -> rechazo cancelacion
E14 PASA - confirmar, enviar y devolver -> exito
E15 PASA - confirmar, enviar y entregar -> exito
15/15 escenarios cumplen su objetivo
TODOS PASAN
```

Figura 3. Resultado de python ejecutar_escenarios.py.

Evidencia 3: Pruebas automáticas (pytest)

```
(.venv) PS C:\Users\anngi\Downloads\EQ_4_UNIDAD4_CLNEG\EQ#4_UNIDAD4_CLNEG> python -m pytest -q
.....................................................................................................  [ 61%]
.....................................................................                                  [100%]
=============================================== warnings summary ===============================================
.venv\Lib\site-packages\fastapi\testclient.py:1
  C:\Users\anngi\Downloads\EQ_4_UNIDAD4_CLNEG\EQ#4_UNIDAD4_CLNEG\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using `httpx`
 with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
181 passed, 1 warning in 1.67s
(.venv) PS C:\Users\anngi\Downloads\EQ_4_UNIDAD4_CLNEG\EQ#4_UNIDAD4_CLNEG>
```

Figura 4. Resultado de python -m pytest -q: 181 passed y 1 advertencia de deprecación de httpx.

Evidencia 4: Swagger: operación exitosa

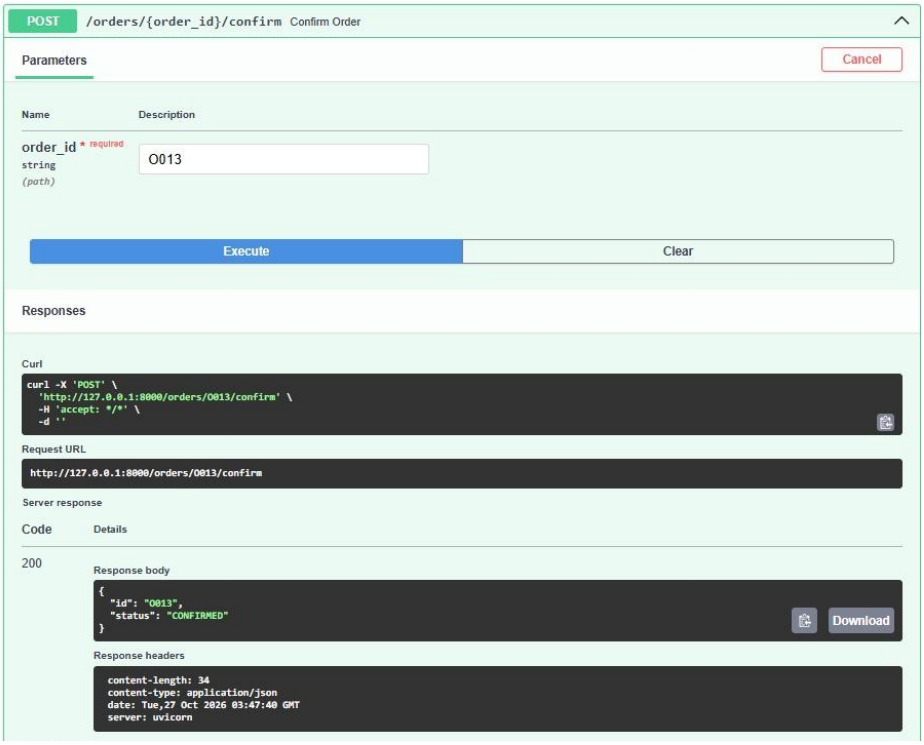


Figura 5. POST /orders/O013/confirm: 200 y estado CONFIRMED.

Evidencia 5: Swagger: operación rechazada

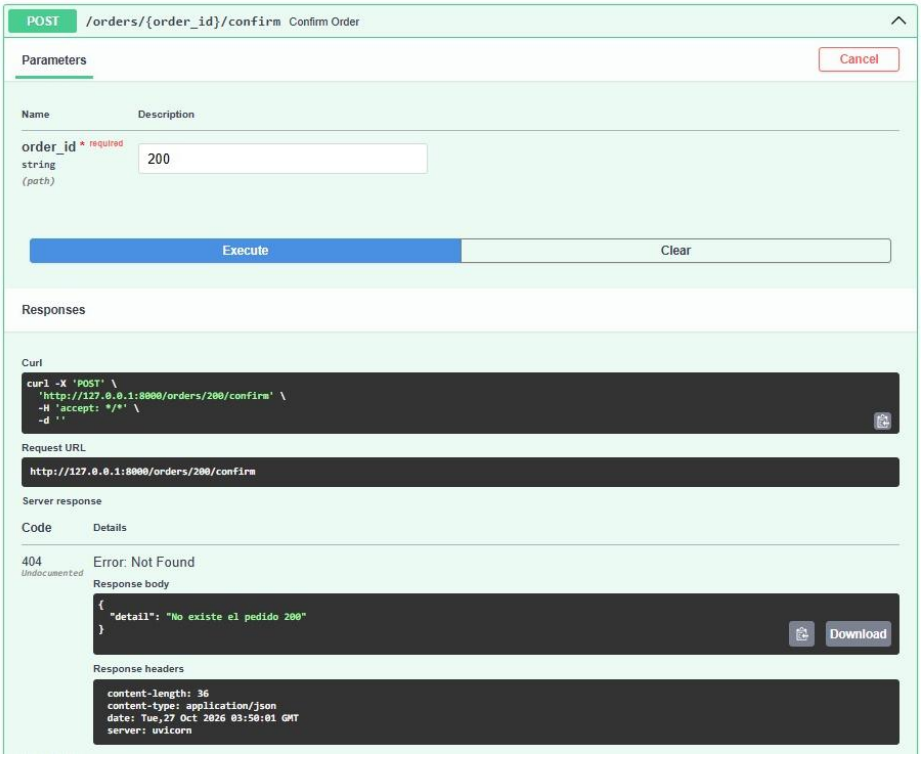


Figura 6. Confirmar un identificador inexistente: 404 con el detalle «No existe el pedido 200».

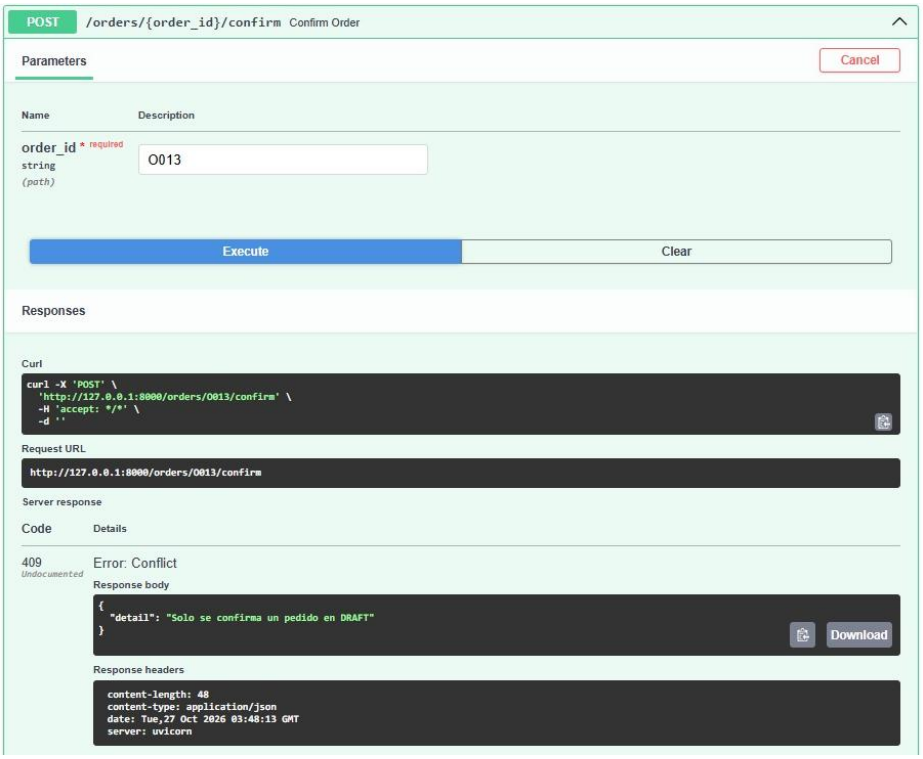


Figura 7. POST /orders/O013/confirm por segunda vez: 409 con el detalle «Solo se confirma un pedido en DRAFT».

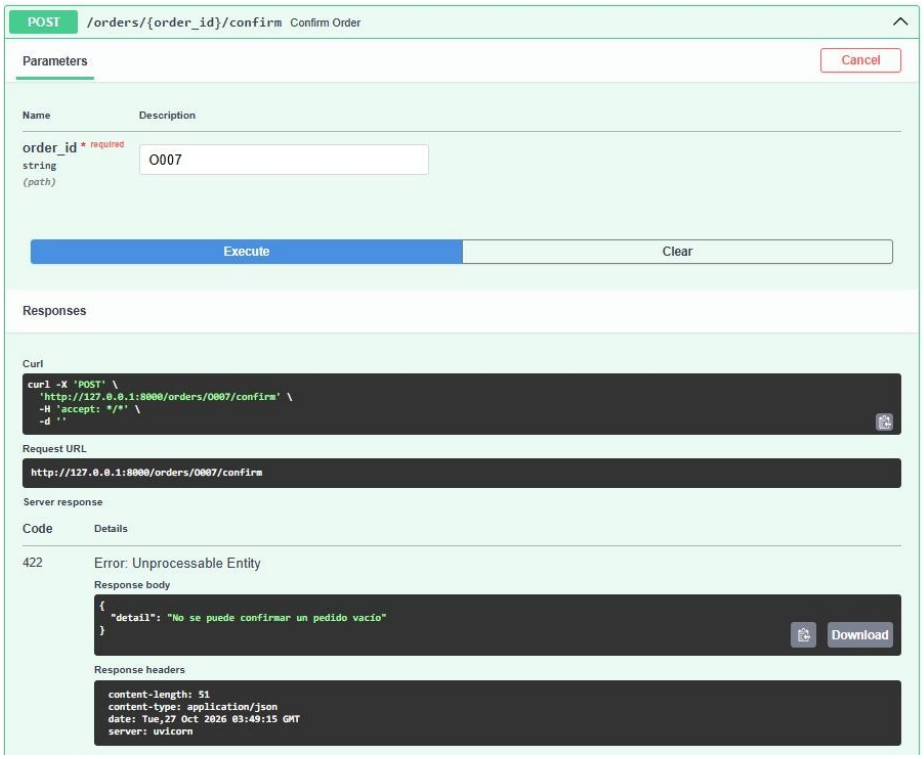


Figura 8. POST /orders/O007/confirm: 422 con el detalle «No se puede confirmar un pedido vacío».

Las tres respuestas rechazadas coinciden con el manejo de excepciones de la API: un identificador inexistente produce 404 (Figura 6); confirmar O013 por segunda vez produce 409 porque el pedido ya no está en DRAFT (InvalidOrderTransition, Figura 7); y confirmar O007, que no tiene artículos, produce 422 (InvalidOrderData, Figura 8). Los casos 409 y 422 también están cubiertos por las pruebas automáticas test_confirm_doble_409_sin_evento_extra y test_confirm_pedido_vacio_422.

6. Reflexión: tres errores encontrados

Error	Qué pasó y cómo se resolvió	Aprendizaje
1. Python no estaba instalado (entorno)	La terminal no reconocía python. Se instaló desde python.org marcando «Add python.exe to PATH» y se verificó con python --version.	Verificar el entorno antes de programar.

Error	Qué pasó y cómo se resolvió	Aprendizaje
2. uvicorn: «Got unexpected extra argument (app.api:app)» (ejecución)	El comando se pegó con espacios o caracteres extra. Se resolvió escribiéndolo a mano como python -m uvicorn app.api:app --reload.	Leer el mensaje completo; copiar y pegar puede arrastrar caracteres invisibles.
3. En Swagger se escribió 200 en order_id y devolvió 404 (uso)	Se confundió el código de respuesta con el identificador. La API buscó un pedido «200» y respondió 404 correctamente. Se corrigió escribiendo O013 (200).	order_id es un dato de entrada, y ese 404 confirma el manejo de identificadores inexistentes.

7. Conclusión General

Separar el dominio de la API permitió que las reglas de negocio (límite de 20 unidades, umbrales estrictos de descuento y transiciones de estado) se probaran sin levantar el servidor y que FastAPI solo tradujera excepciones a códigos HTTP (404, 409 y 422). Las Specifications dejaron cada condición del descuento con nombre propio y combinable con &, | y ~, y Money con Decimal evitó los errores de float en las comparaciones contra 500 y 5000. Registrar los eventos solo después de una transición válida mantiene consistente el historial del pedido, como se vio con la confirmación doble de O013.

Conclusiones individuales:

- **García Cerda Randal Guillermo (2112493 ITS)**

La realización de esta práctica me permitió comprender mejor la importancia de separar las reglas de negocio de la API para facilitar el desarrollo, las pruebas y el mantenimiento del sistema. Aprendí que mantener esta separación permite verificar las condiciones de cada operación sin necesidad de iniciar el servidor, lo que ayuda a identificar errores de manera más sencilla. También comprendí

la importancia de establecer límites y validaciones para evitar que se realicen operaciones que no cumplen con las condiciones establecidas para los pedidos.

- **Sarmiento Hernández Cesar Martín (2119427 ITS)**

A través de esta actividad aprendí a utilizar Specifications para organizar las condiciones de los descuentos de una manera más clara, permitiendo que cada regla pudiera identificarse y combinarse con otras según las necesidades del sistema. Asimismo, comprendí la importancia de utilizar Decimal para trabajar con cantidades monetarias, ya que permite realizar comparaciones más precisas que el tipo de dato float. Estos conocimientos me ayudaron a entender cómo implementar reglas de negocio de forma ordenada y reducir posibles errores en los cálculos y las validaciones.

- **Torres Pasillas Angela Susana (2101844 ITS)**

Esta práctica me permitió comprender la importancia de controlar correctamente las transiciones de estado de un pedido y registrar sus eventos únicamente cuando una operación se completa de manera válida. También aprendí que es necesario evitar que una misma acción se procese varias veces cuando no corresponde, ya que esto podría generar inconsistencias en el historial. Finalmente, entendí cómo el manejo de excepciones y su correspondencia con los códigos HTTP permite comunicar los errores de forma adecuada, manteniendo separadas las responsabilidades de la lógica de negocio y la API.

8. Preguntas del reporte y respuestas

Pregunta 1. ¿Por qué se usa Decimal en lugar de float para precios?

float guarda los números en binario y muchos decimales no tienen representación exacta (0.1 + 0.2 da 0.30000000000000004). En dinero esos errores se acumulan y pueden alterar comparaciones con umbrales como 500. Decimal es exacto en base decimal, por eso Money rechaza float.

Pregunta 2. ¿Cuál es la diferencia entre contar renglones y sumar unidades?

Un renglón es una línea del pedido; las unidades son la suma de las cantidades. O001 tiene 2 renglones y 4 unidades; O008 tiene 1 renglón con 20 unidades. El límite de 20 y el mínimo de 4 hablan de unidades, por eso Order.units usa la suma de cantidades.

Pregunta 3. ¿Qué ventaja tiene encapsular una regla en Specification?

Cada regla tiene nombre, se prueba por separado, se combina con &, | y ~ sin repetir condiciones y no modifica el pedido. Cambiar un umbral implica editar una sola clase.

Pregunta 4. ¿Por qué los eventos se registran solo después de una transición válida?

Un evento afirma que algo ya ocurrió. Registrarlo antes de validar dejaría eventos falsos tras operaciones rechazadas y otros componentes reaccionarían a un hecho inexistente.

Pregunta 5. ¿Qué parte del código se puede reutilizar si se reemplaza FastAPI?

Todo el dominio, las Specifications, pricing y cargar_csv.py, porque no dependen de FastAPI. Solo app/api.py, que traduce HTTP y excepciones a códigos de estado, se reemplazaría. ejecutar_escenarios.py ya es una segunda interfaz que usa el dominio sin FastAPI.

9. Repositorio

https://github.com/Angietptp/CLNEG_EQ-4_UNIDAD4